DOCUMENTATION

Getting Started

AI-Powered Investor Intelligence Platform — a beginner-friendly guide to running this project: every Azure service explained, local setup, testing, deployment, and real troubleshooting.

---

github.com/AzaRKazar/AI-Powered-Investor-Intelligence-Platform
Companion documents: architecture.md (system design) · build-story.md (the narrative)

Getting Started

This guide is for someone new to this repository who wants to actually run it — locally, with Docker, or fully deployed to Azure — and understand *why* each piece exists, not just which command to type. If you already know your way around FastAPI/React/Azure, the main README is faster. This document is the slow, patient version, with links out to the official docs for every technology involved.

For *why* things were built this way, see build-story.md. For the technical reference (diagrams, schemas, API contracts), see architecture.md.

---

1. What this project actually is, in plain terms

You upload a PDF (a company's annual report — a "10-K" in US filing terminology). The system:

1. Turns the PDF into plain text.
2. Splits that text into small, meaningful chunks and turns each chunk into a list of numbers (an "embedding") that captures its meaning.
3. Stores those chunks so they can be searched by *meaning*, not just keyword matching.
4. Asks an LLM (a large language model — think ChatGPT, but Azure's hosted version) to read the relevant chunks and pull out specific numbers: revenue, net income, and so on.
5. Saves those numbers to a database.
6. Lets you browse the results on a dashboard, and ask follow-up questions in a chat panel — the chat also searches the stored chunks for relevant context before answering, which is why this pattern is called **RAG** (Retrieval-Augmented Generation).

Everything in the sections below exists to make one of those six steps possible.

---

2. Prerequisites — tools to install first

Tool	What it's for	Get it
Python 3.12+	Runs the backend	python.org/downloads
uv	Fast Python package manager (this project doesn't use plain `pip`)	docs.astral.sh/uv
Node.js 20+	Builds the React frontend	nodejs.org
Docker	Runs a local Postgres database, and builds the deployable image	docs.docker.com/get-started
Git	Clones the repo	git-scm.com
Azure CLI (`az`)	Only needed if you're setting up your own Azure resources	learn.microsoft.com/cli/azure/install-azure-cli
kubectl	Only needed if you're deploying to Kubernetes	kubernetes.io/docs/tasks/tools

You don't need an Azure account at all to read the code — but to actually *run* it (even locally), you need at minimum a Postgres database, an Azure AI Search resource, and an Azure OpenAI resource, because the app calls out to real cloud services rather than running any AI models locally. Section 3 walks through creating each one.

---

3. Setting up the cloud services

This is the part that trips up most beginners, so it gets the most detail. Every service below is created through the **Azure Portal** (portal.azure.com) — sign in, then use the search bar at the top to find each service by name.

First: create a **Resource Group** to hold everything (Portal → search "Resource groups" → Create). Think of it as a folder — it makes it trivial to delete every resource for this project in one action later, which matters a lot for cost control (see Section 8).

3.1 Azure Database for PostgreSQL — Flexible Server

What it is: a managed relational (SQL) database. This project uses it to store the final extracted KPI numbers — one row per company, per year.

Why this project uses it: the extracted numbers are structured (revenue, net income, etc.) and need to be queried and displayed reliably — a normal SQL database is the right tool for that, as opposed to the vector search database (3.2), which is for *unstructured* text.

How to create it:

1. Portal → **Create a resource** → search **"Azure Database for PostgreSQL flexible servers"** → Create

2. Choose your resource group and region
3. Compute + storage: pick **Burstable**, smallest size (**B1ms**) — cheap and plenty for this project's scale
4. Set an admin username and password — you'll need both for the `.env` file later
5. Networking: allow public access, and add a firewall rule for your own current IP address (the Portal usually offers to add this automatically)
6. Review + create

What you'll need from it: hostname (e.g. `yourserver.postgres.database.azure.com`), port (`5432`), admin username, admin password, and a database name (create one, e.g. `investor_intelligence`).

Learn more: learn.microsoft.com/azure/postgresql/flexible-server · postgresql.org/docs for SQL itself.

3.2 Azure AI Search

What it is: a search engine that supports both traditional keyword search *and* **vector search** (searching by meaning, using those embeddings mentioned in Section 1) — combining both is called **hybrid search**.

Why this project uses it: after a PDF is chunked, every chunk gets stored here. When the app needs to answer "what are Apple's risk factors," it doesn't re-read the whole PDF — it asks this service for the most relevant chunks first.

How to create it:

1. Portal → **Create a resource** → search **"Azure AI Search"** → Create
2. Choose your resource group and region
3. Pricing tier: **Free (F0)** — sufficient for this project, and there's no cost
4. Review + create

What you'll need from it: the endpoint URL (Overview page) and an admin API key (Settings → Keys). The actual search *index* (the schema — what fields each stored chunk has) gets created automatically by this project's own code (`vectorstore/create_index.py`) the first time the app starts — you don't need to configure that by hand.

Learn more: learn.microsoft.com/azure/search · for the concept of vector search generally, learn.microsoft.com/azure/search/vector-search-overview.

3.3 Azure OpenAI

What it is: Microsoft's hosted version of OpenAI's models (GPT, embeddings) — same underlying models as ChatGPT, but running inside Azure's infrastructure with Azure-style billing and access control.

Why this project uses it: this is the actual "AI" — it reads retrieved text chunks and extracts structured KPIs, and separately, it answers chat questions. It also generates the embeddings mentioned in 3.1/3.2.

How to create it — the part beginners get stuck on: Azure OpenAI access is **not automatically available** on every subscription the way most Azure services are. You typically need to request access and be approved before you can create a deployment, and depending on your subscription type (a free trial vs. a Pay-As-You-Go account with billing enabled), you may hit a block requiring you to upgrade your subscription first. This is a real, known friction point — budget extra time for it, and don't assume something's broken if it doesn't work instantly.

1. Portal → **Create a resource** → search **"Azure OpenAI"** → Create (if you don't see access, you may need to request it first via the same search result's information page)
2. Choose your resource group and region (not every region has every model — `East US` is a safe, commonly-available choice)
3. Once created, go to **Azure AI Foundry** (the model deployment interface, linked from the resource's Overview page) and deploy two separate models:
   - A chat model (this project uses `gpt-5-mini`) — give the deployment a name, you'll reference it by that name
   - An embedding model (this project uses `text-embedding-ada-002`) — same, name it
4. From the resource's **Keys and Endpoint** page, grab the endpoint URL and an API key

What you'll need from it: endpoint, API key, API version (a date string Azure uses to version its API, e.g. `2024-10-21`), and the two deployment names you chose.

Cost note: this is pay-per-use, not a flat fee — you're billed per token processed. At this project's scale (a handful of documents), real-world cost has been well under a dollar total.

Learn more: learn.microsoft.com/azure/ai-services/openai · platform.openai.com/docs for the underlying model APIs themselves (Azure's API shape closely mirrors OpenAI's own).

3.4 Azure Document Intelligence *(optional)*

What it is: an OCR (optical character recognition) service — it reads text out of images, including scanned documents.

Why this project uses it: most PDFs have a real, selectable text layer that's easy to extract from directly. Some don't — a scanned or print-flattened PDF is really just a picture of a page, with no underlying text at all. This project detects that case and falls back to this service to recover the text via OCR instead of silently returning nothing.

Do you need it? Only if you plan to upload a scanned PDF. The app runs completely fine without it for any normal, text-based PDF.

How to create it:

1. Portal → **Create a resource** → search **"Document Intelligence"** → Create
2. Pricing tier: **Free (F0)** covers light use (500 pages/month); if you hit a file-size limit on a large scanned PDF, switch to **Standard (S0)** — see the Continuous Deployment section of the README for a real example of this happening
3. Grab endpoint + key from **Keys and Endpoint**

Learn more: learn.microsoft.com/azure/ai-services/document-intelligence

3.5 Azure Container Registry (ACR) *(only needed to deploy)*

What it is: a private storage location for Docker container images — like Docker Hub, but private and inside your own Azure account.

Why this project uses it: to run the app on Kubernetes (3.6), Kubernetes needs to pull the built container image from somewhere. This is that somewhere.

How to create it: Portal → **Create a resource** → search **"Container Registry"** → Create → pick **Basic** tier. Enable **Admin user** (Settings → Access keys) if you want simple username/password push access rather than Azure AD-based auth.

Learn more: learn.microsoft.com/azure/container-registry · docs.docker.com for Docker/container concepts generally.

3.6 Azure Kubernetes Service (AKS) *(only needed to deploy)*

What it is: a managed Kubernetes cluster — Kubernetes is a system for running containers reliably at scale (restarting them if they crash, load-balancing traffic to them, etc.).

Why this project uses it: it's how the app is actually hosted live, publicly reachable, rather than just running on someone's laptop.

How to create it: Portal → **Create a resource** → search **"Kubernetes Service"** → Create. Use a single small node (this project uses `Standard_D2as_v7`), and on the **Integrations** tab, attach the ACR from 3.5 so the cluster can pull images from it without extra configuration later. Full deploy steps (building the image, applying manifests, getting the app's public URL) are in the main README's Deploying to Azure section.

Important cost note: unlike everything else above, AKS and Postgres **bill continuously while running**, not just per-use. This project's own operating discipline is to delete AKS (not just stop it — see Section 8) and stop Postgres between work sessions.

Learn more: learn.microsoft.com/azure/aks · kubernetes.io/docs/concepts for Kubernetes concepts independent of Azure.

---

4. Running the project locally

Once you have Postgres, AI Search, and Azure OpenAI credentials from Section 3:

```
# 1. Clone the repo
git clone https://github.com/AzaRKazar/AI-Powered-Investor-Intelligence-Platform.git
cd AI-Powered-Investor-Intelligence-Platform

# 2. Backend dependencies
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv
source .venv/bin/activate      # on Windows: .venv\Scripts\activate
uv pip install -r requirements.txt

# 3. Frontend dependencies
cd frontend && npm install && cd ..

# 4. Environment variables
cp .env.example .env
# now open .env and fill in every value with your real credentials from Section 3
```

`.env.example` lists every variable the app reads, with a comment above each explaining what it's for — treat it as the authoritative list, since code changes over time and this guide might not always be perfectly in sync with it.

Running it, two ways:

- **Frontend development mode** (hot reload while you edit React code) — run two terminals: `bash # terminal 1 python app.py # terminal 2 cd frontend && npm run dev` Visit `http://localhost:5173`.
- **Production-like mode** (one process, exactly how Docker runs it): `bash cd frontend && npm run build && cd .. python app.py` Visit `http://localhost:8000`.

Optional — local Postgres instead of Azure Postgres, useful if you don't want to touch the cloud database while just poking around the code: `docker compose up -d postgres` starts a local Postgres container matching the schema this project expects (see `docker-compose.yml`).

---

5. Running the test suite

```
uv pip install -r requirements-dev.txt
pytest -v
```

22 tests, covering the LangGraph extraction pipeline's retry/validation logic, retrieval deduplication, PDF text-detection, and OCR error-handling. None of them make live calls to Azure — the LLM, the search retriever, and the OCR client are all mocked, but grounded in real annual-report data already committed to the repo (`data/markdown/`). This is the same command CI runs automatically on every push.

Docs: pytest.org

---

6. Running with Docker

```
docker compose up -d
```

This builds the full app (a multi-stage build — it compiles the React frontend first, then copies the result into the Python image, so you don't need Node installed at all if you only want to run via Docker) and starts it alongside a local Postgres container. You still need real Azure AI Search / OpenAI credentials in `.env` — Docker only replaces the Postgres and hosting pieces, not the AI services.

Docs: docs.docker.com/compose

---

7. Deploying to Azure

Two ways, both documented in detail in the main README:

- **Manual**: README § Deploying to Azure — build the image, push to ACR, apply the Kubernetes manifests yourself.
- **Automated**: README § Continuous Deployment — a GitHub Actions workflow does the same steps on a button-press (`workflow_dispatch`), once you've set the required repository secrets.

---

8. Troubleshooting — real issues, not hypotheticals

Everything in this section actually happened during this project's development, not a generic checklist:

- **"Rate limit exceeded" during PDF ingestion.** `SemanticChunker` embeds nearly every sentence individually, which can exceed a freshly-created Azure OpenAI deployment's low default rate limit. Fixed in the code already (`chunk_size=16`, `max_retries=10` on the embeddings client) — if you still hit this, your deployment's quota may need raising in the Azure Portal (AI Foundry → your deployment → quota settings).
- **KPI fields come back `null` for a PDF you just uploaded.** Two possible causes: (a) the model genuinely couldn't find that field in the retrieved context after 3 attempts — check the server logs for a line like `Extraction incomplete for X - still missing: ...`, which means the pipeline tried honestly and gave up, not a bug; or (b) the PDF is scanned/image-only with no text layer at all — see Section 3.4.
- **Postgres connection times out.** Almost always a firewall rule — Azure Postgres only accepts connections from IP addresses you've explicitly allowed. If your own internet connection's IP address changes (common on a laptop switching networks), you'll need to add a new firewall rule for the current IP in the Portal.

- **AKS pods stuck in `ImagePullBackOff`.** The cluster doesn't have permission to pull from your Container Registry. Fix: `az aks update --resource-group <rg> --name <cluster> --attach-acr <registry-name>`.
- **A freshly-deployed pod won't start at all.** Check whether the `investor-intel-secrets` Kubernetes Secret exists (`kubectl get secret investor-intel-secrets`) — it holds all the `.env`-equivalent values and isn't created automatically by the deploy workflow.
- **You stopped AKS but you're still being billed.** `az aks stop` only pauses the compute (the node VMs) — it does *not* stop the Load Balancer or Public IP address(es) AKS creates alongside the cluster, and those bill hourly regardless of whether the cluster is running. If you're done for a while, **delete** the cluster (`az aks delete`) rather than just stopping it.

---

9. Further learning resources

If a piece of this project is unfamiliar and you want to actually understand it rather than just run it:

Topic	Where to start
FastAPI (the backend framework)	fastapi.tiangolo.com — their own tutorial is excellent and hands-on
React	react.dev/learn
TypeScript	typescriptlang.org/docs
LangGraph (the state-machine orchestration library)	langchain-ai.github.io/langgraph
RAG (Retrieval-Augmented Generation) as a concept	learn.microsoft.com/azure/search/retrieval-augmented-generation-overview
SQL / PostgreSQL	postgresql.org/docs
Docker	docs.docker.com
Kubernetes	kubernetes.io/docs/concepts
GitHub Actions (CI/CD)	docs.github.com/actions
pytest	docs.pytest.org

Still stuck on something not covered here? Open an issue on the repository, or start from architecture.md to see how the pieces fit together structurally.